

Day: / /

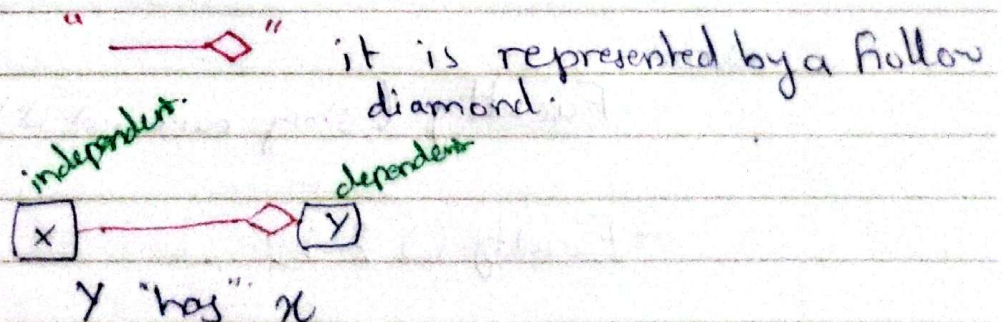
Date: / /

## Aggregation:

In this we use simple objects inside complex objects but are not responsible for them.

- It is "Has" or "uses a" relationship
- It is ~~the~~ a very weak relationship and there is no ownership
- In this a <sup>dependent</sup> class contains another independent class but it is **NOT** responsible for its **life line**.
- The dependent object does **NOT** create or destroy the independent object, so even if the ~~at~~ dependent object is destroyed, independent remains.

## Symbols



Yousaf



Day:

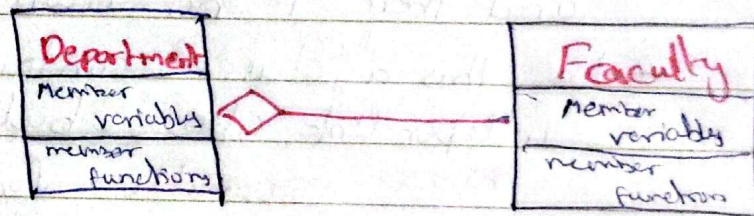
Date: / /

Examples :

1) First way to do aggregation:

- First way is to use a pointer.

classes I will be developing:



```
class Faculty {
```

```
    string name;
    int id;
```

```
public:
```

```
    Faculty ( string name , int id ) : name(name) , id(id) {}
```

```
    ~Faculty () {}
```

```
};
```

Yousaf



Day:

Date: / /

```
class Department {
```

```
    int size ;  
    Faculty** ptr;
```

```
public:
```

```
    Department (int s=3) : size(s) {
```

```
        Faculty ptr = new Faculty* [size];  
    }
```

```
    void add Faculty (Faculty* f, int Index) {
```

```
        ptr[Index] = f;  
    }
```

```
    ~ Department () {
```

```
        delete [] ptr;  
    }
```

```
};
```

```
int main () {
```

```
    Faculty f1 ("Harder", 24), f2 ("Ali", 92);
```

```
    Department d(2);
```

```
    d.add Faculty (&f1);
```

```
    d.add Faculty (&f2);
```

```
}
```

Yousaf



## Explanation:

Inside Department and @ side the add faculty function:

we are not creating any new faculty we are just pointing faculty towards that.

This is aggregation.

we are not creating the faculty rather we are just using it.

In destructor we only destroy ptr array which is an array of pointers there is no value, which is being destroyed.

- Inside main we have to send addresses because in add faculty we require \*pointers as parameters.

Day:

Date: / /

2) 2nd way to do aggregation.

The second way is to use an alias (&) variable.

Using the same Faculty class.

```
class Department {  
    Faculty Faculty  
    Faculty & Fac;
```

public:

```
    Department (Faculty & Fac) : Fac(F) {}  
};
```

Explanation:

This code now takes in a Faculty object and makes an alias.

now in main we only will pass object, no pointer, no reference as inside parameter we will get the reference.

- This was the code B, only one faculty member.

- This only works for single because an array of reference is not possible.

Yousaf



Date: / /

- Also very important;

we have to initialize the reference variable in member initializer as it cannot be created without initializing.

- Do examples on your own or look at example 6 from compilation